

Métodos factory en Java y generación de aleatorios

¿Que es el método factory?

Los métodos factory en Java son una técnica de diseño creacional, a menudo implementada como métodos estáticos o a través del patrón *Factory Method*, que encapsula la lógica de instanciación de objetos. Permiten devolver instancias de una clase, subclasses o interfaces sin especificar la clase concreta, lo que mejora la flexibilidad, desacopla el código cliente y facilita el mantenimiento.

Diferencias entre Factory method y Factory simple

Existen 2 tipos de métodos factory

Simple Factory es un patrón de diseño que consiste en una única clase que se encarga de crear y devolver diferentes tipos de objetos, según el parámetro que recibe.

Factory Method es un patrón de diseño que consiste en definir un método abstracto en una clase padre, y que sean las subclasses concretas las que implementen cómo crear cada tipo específico de objeto.

Es decir tenemos un método factory simple con el cual tenemos una clase que con ciertos parámetros va a crear los objetos diferentes y en el factory method vamos a querer una clase propia a la que le vamos a delegar la creación de cada objeto diferente es decir si queremos crear objetos diferentes vamos a ir añadiendo diferentes clases.

Factory simple -> Una fábrica que crea todo tipo de objetos

Factory method -> Muchas fábricas diferentes una por cada tipo de objeto que queramos crear

Factory simple como funciona

Paso 1:

Establece un contrato común que tendrán todos los objetos que la fábrica puede crear. Esto permite que el cliente trabaje con una abstracción y no dependa de implementaciones concretas.

Paso 2:

Desarrolla las clases concretas que implementan la interfaz base. Cada una tendrá su propia lógica y comportamiento específico, pero todas cumplen con el contrato definido en el paso anterior.

Paso 3:

Construye una única clase que contendrá un método encargado de crear objetos. Este método recibe un parámetro (como un texto o un número) y, mediante condicionales, decide qué tipo de objeto concreto debe instanciar y devolver.

Paso 4:

El método de la fábrica debe retornar el objeto creado utilizando el tipo de la interfaz base, no el tipo concreto. De esta forma, el cliente que recibe el objeto solo conoce la interfaz y no los detalles internos de la implementación.

Paso 5:

El cliente, en lugar de crear objetos directamente con `new`, solicita los objetos a la fábrica pasando el parámetro correspondiente. Luego trabaja con ellos a través de la interfaz base.

Factory method como funciona

Paso 1:

Crear una clase base. Esta representa a los tipos de datos regresados por la fábrica. La clase puede ser concreta, pero lo más común y recomendable es que se use una clase abstracta o una interface para realmente facilitar un bajo acoplamiento.

Paso 2:

Crear una serie de clases hija que extiendan (o implementen) la clase base. Claro que al final las clases tendrán detalles diferentes en la implementación. Estas clases serán los objetos creados por nuestras fábricas.

Paso 3:

Crear una clase base Factory Method. Esta puede ser una clase abstracta o una interface, con un método que regrese una instancia de la clase base. Este método puede recibir parámetros si es necesario para que decida cuál es la instancia que se debe regresar.

Paso 4:

Crear una o más implementaciones de la clase Factory. Estas implementaciones serán quienes regresen los objetos concretos creados por la fábrica. Si necesitamos más de una forma de crear los objetos, podemos crear también más de una implementación de la fábrica.

Paso 5:

Obtener la instancia que se regresará al cliente. La instancia puede ser creada de inmediato, tomada de un pool de objetos, u obtenido de cualquier otra forma que sea necesaria. La manera en la que el Factory Method cree la instancia dependerá de las necesidades particulares de tu aplicación.

Paso 6:

Regresar al cliente la instancia como una referencia del tipo de la clase base.

SIMPLE FACTORY

Ventajas

1. Simplicidad y facilidad de implementación

Es el patrón más sencillo de entender y aplicar. Con una sola clase y un método se puede implementar completamente.

2. Centralización de la lógica de creación

Toda la responsabilidad de crear objetos está en un único lugar, lo que facilita encontrar y modificar el código de creación si es necesario.

3. Bajo acoplamiento entre cliente y productos concretos

El cliente solo conoce la interfaz base y la fábrica, no las clases concretas que se instancian.

4. Ideal para proyectos pequeños o con pocos productos

Cuando la aplicación no tiene muchos tipos de objetos que crear, este patrón es más que suficiente y evita sobreingeniería.

Desventajas

1. Viola el principio Abierto/Cerrado (Open/Closed Principle)

Cada vez que se añade un nuevo tipo de producto, es necesario modificar la clase fábrica para agregar un nuevo condicional. Esto abre la puerta a posibles errores en código existente.

2. Puede volverse difícil de mantener

Si el número de productos crece demasiado, el método de creación se llena de condicionales y se vuelve difícil de leer y mantener.

3. No es fácil de extender sin modificar

La única forma de cambiar el comportamiento de creación es modificando la clase fábrica, no se puede extender mediante herencia o composición.

4. Mezcla responsabilidades

La clase fábrica tiene tanto la responsabilidad de decidir qué producto crear como la de crearlo, lo que puede considerarse una violación del principio de responsabilidad única.

FACTORY METHOD

Ventajas

1. Cumple con el principio Abierto/Cerrado

Para añadir un nuevo producto, solo hay que crear una nueva subclase fábrica. No es necesario modificar el código existente, lo que reduce el riesgo de introducir errores.

2. Mayor flexibilidad y extensibilidad

Cada subclase fábrica puede decidir cómo crear su producto específico, permitiendo diferentes formas de instanciación sin afectar a otras fábricas.

3. Separa la lógica de creación por producto

Cada tipo de producto tiene su propia fábrica, lo que hace que el código esté más organizado y sea más fácil de localizar y modificar.

4. Permite diferentes variantes de creación

Si se necesitan diferentes formas de crear un mismo tipo de objeto, se pueden crear múltiples implementaciones de fábrica sin afectar al cliente.

5. Facilita el testing

Al tener fábricas separadas, se pueden probar de forma independiente y se pueden crear fábricas mock para pruebas unitarias.

Desventajas

1. Aumenta la complejidad del código

Requiere crear una jerarquía de clases fábrica, lo que incrementa el número de clases y hace que el código sea más difícil de seguir para desarrolladores novatos.

2. Puede resultar en sobreingeniería

Para proyectos pequeños o con pocos productos, Factory Method introduce una complejidad innecesaria que Simple Factory resolvería con menos código.

3. Mayor número de clases a mantener

Por cada nuevo producto concreto, se necesita crear también una nueva subclase fábrica, duplicando el número de clases a mantener.

4. Dificulta la comprensión inicial

Para alguien que no está familiarizado con el patrón, entender el flujo de creación a través de múltiples clases puede resultar confuso.

5. No es adecuado para todos los escenarios

Si los productos no varían con frecuencia o si la lógica de creación es muy simple, Factory Method puede ser excesivo.

Generación de Aleatorios en Java

¿Qué es la generación de aleatorios?

La generación de aleatorios en Java es el proceso mediante el cual se obtienen valores que aparentan ser impredecibles o sin un patrón definido, como números, booleanos o valores dentro de un rango específico. En realidad, estos valores son generados por algoritmos deterministas llamados generadores de números pseudoaleatorios, que producen secuencias de números que simulan ser aleatorias.

Principales formas de generar aleatorios en Java

1. Clase `Math.random()`

Es el método más simple y directo. Devuelve un valor `double` entre 0.0 (incluido) y 1.0 (excluido).

Características:

- Fácil de usar para casos simples
 - No permite personalizar la semilla o el algoritmo
 - Útil cuando solo se necesita un valor aleatorio básico
-

2. Clase `Random (java.util.Random)`

Es la forma más tradicional y versátil. Permite generar diferentes tipos de datos aleatorios.

Características:

- Permite establecer una semilla para reproducir secuencias
- Ofrece métodos específicos para enteros, doubles, booleanos, etc.
- Se puede crear una instancia y reutilizarla para múltiples generaciones

Métodos principales:

- `nextInt()`: entero cualquiera
 - `nextInt(int bound)`: entero entre 0 y el límite especificado (excluido)
 - `nextDouble()`: double entre 0.0 y 1.0
 - `nextBoolean()`: true o false
 - `nextLong()`: long aleatorio
-

3. Clase `ThreadLocalRandom` (`java.util.concurrent.ThreadLocalRandom`)

Es una variante moderna diseñada para entornos con múltiples hilos (multithreading).

Características:

- Mejor rendimiento en aplicaciones concurrentes
 - No requiere crear una instancia, se usa directamente
 - Evita problemas de contención entre hilos
 - Sintaxis más limpia que `Random`
-

4. Clase `SecureRandom` (`java.security.SecureRandom`)

Es la opción para aplicaciones que requieren seguridad, como generación de contraseñas, claves criptográficas o tokens.

Características:

- Genera números con mayor imprevisibilidad
- Utiliza algoritmos criptográficamente seguros
- Es más lento que `Random` pero necesario para contextos de seguridad
- Ideal para datos sensibles que no deben ser predecibles

Instrucciones para la Generación de Aleatorios en Java

Paso 1:

Elegir la clase adecuada según la necesidad del programa. Si solo se necesita un valor aleatorio simple se puede usar `Math.random()`. Si se requiere generar diferentes tipos de datos o establecer una semilla se debe usar la clase `Random`. Si la aplicación trabaja con múltiples hilos concurrentes es recomendable usar `ThreadLocalRandom`. Si los valores aleatorios se utilizarán para fines de seguridad como contraseñas o claves se debe usar `SecureRandom`.

Paso 2:

Importar la clase correspondiente al inicio del archivo. Para `Math.random()` no es necesario importar nada porque pertenece al paquete `java.lang`. Para `Random` se debe importar `java.util.Random`. Para `ThreadLocalRandom` se debe importar `java.util.concurrent.ThreadLocalRandom`. Para `SecureRandom` se debe importar `java.security.SecureRandom`.

Paso 3:

Crear una instancia del generador si es necesario. `Math.random()` no requiere instancia porque es un método estático. Para `Random` se crea un objeto con `new Random()` y opcionalmente se puede pasar una semilla como parámetro en el constructor. Para `ThreadLocalRandom` no se crea instancia directamente sino que se obtiene mediante el método estático `ThreadLocalRandom.current()`. Para `SecureRandom` se crea una instancia con `new SecureRandom()`.

Paso 4:

Definir el rango o el tipo de valor que se desea generar. Para números enteros se debe especificar si se necesita un valor sin límite o dentro de un rango específico. Para números decimales se debe considerar si se requiere precisión float o double. Para valores booleanos se genera directamente verdadero o falso. Para generación de bytes se debe definir un arreglo donde se almacenarán los valores generados.

Paso 5:

Utilizar el método correspondiente del generador para obtener el valor aleatorio. Para `Math.random()` se obtiene un double entre 0.0 y 1.0. Para `Random` se dispone de métodos como `nextInt()`, `nextInt(int bound)`, `nextDouble()`, `nextBoolean()`, `nextLong()` entre otros. Para `ThreadLocalRandom` se utilizan métodos similares pero accediendo a través de `current()`. Para `SecureRandom` se utilizan métodos como `nextInt()`, `nextBytes()` o `generateSeed()`.

Paso 6:

Almacenar el valor generado en una variable del tipo adecuado y utilizarlo dentro del programa según sea necesario, ya sea para cálculos, selección de elementos, simulación de eventos, generación de identificadores o cualquier otro propósito que requiera valores impredecibles.

Paso 7:

Si se necesita reproducir la misma secuencia de valores aleatorios en ejecuciones futuras, se debe establecer una semilla fija al crear el generador. Para `Random` se pasa la semilla al constructor o se usa el método `setSeed()`. Para `SecureRandom` se puede usar `setSeed()` aunque esto puede reducir su seguridad. Para `Math.random()` y `ThreadLocalRandom` no es posible establecer una semilla manualmente.

Ventajas y Desventajas de los Métodos de Generación de Aleatorios en Java

MATH.RANDOM()

Ventajas	Desventajas
Muy simple y fácil de usar	Solo genera double (0.0 a 1.0)
No requiere importar clases	No permite establecer semilla
Código conciso (una línea)	No es adecuado para multihilo
Ideal para ejemplos rápidos	No es seguro para datos sensibles

RANDOM (java.util.Random)

Ventajas	Desventajas
Versátil: genera enteros, doubles, booleanos, etc.	Requiere crear una instancia
Permite establecer semilla para reproducir secuencias	Puede tener problemas de contención en multihilo
Métodos específicos para rangos definidos	No es criptográficamente seguro

Ampliamente conocido y documentado

Más líneas de código que `Math.random()`

THREADLOCALRANDOM

Ventajas

Excelente rendimiento en aplicaciones con múltiples hilos

No requiere crear instancia, se usa directamente

Evita problemas de contención entre hilos

Métodos similares a `Random` para todos los tipos

Desventajas

No permite establecer semilla manualmente

Disponible solo desde Java 7

Sintaxis ligeramente diferente a `Random`

Menos conocido que `Random`

SECURERANDOM

Ventajas

Criptográficamente seguro

Ideal para contraseñas, tokens y datos sensibles

Generación impredecible

Desventajas

Rendimiento más lento que las otras opciones

Mayor complejidad de uso

Puede bloquearse si no hay suficiente entropía

Basado en algoritmos robustos

Requiere importar paquete `java.security`
